

Probability Concepts and Random Number Generation

Kapil Dev Pant (5 hrs)

Content

1. Probability Concepts in simulation-Stochastic variable
2. Discrete probability function
3. Continuous probability function
4. Random numbers
5. Properties of Random numbers
6. Pseudo-Random Numbers
7. Techniques for generation of Random numbers
8. Test for random number generation.
 - ▶ Uniformity Test (k-test and chi square test)
 - ▶ Independency test (run test and Auto Correlation test)

Introduction to Probability in Simulation

- ▶ Probability: The measure of the likelihood that an event will occur

It is expressed as a number between 0 and 1:

- ▶ 0 means the event is impossible.
 - ▶ 1 means the event is certain.
- ▶ Example: The probability of flipping a coin and getting heads is 0.5 (or 50%)..

$$\text{Probability (P)} = \frac{\text{Number of favorable outcomes}}{\text{Total number of possible outcomes}}$$

- ▶ Simulation: The process of mimicking real-world processes through mathematical models.
- ▶ Importance in Simulation:
 - ▶ Used to model uncertainty.
 - ▶ Helps in predicting outcomes.
 - ▶ Enhances decision-making under uncertainty.

Applications of Probability in Simulation

Fields of Application:

- ▶ Business: Risk management.
- ▶ Healthcare: Disease spread models.
- ▶ Manufacturing: Process optimization.

Stochastic Variables

A stochastic variable (random variable) is a variable whose value is subject to variations due to randomness.

Types:

- ▶ Discrete Stochastic Variables: Take specific values (e.g., number of heads in a coin toss).
- ▶ Continuous Stochastic Variables: Take any value within a range (e.g., temperature readings).

Applications in Simulation:

- ▶ Weather forecasting.
- ▶ Risk analysis.

Discrete Probability Function

A function that provides the probabilities of occurrence of different possible outcomes for a discrete random variable.

Example:

- ▶ Tossing a fair coin:
 - ▶ Heads: 0.5
 - ▶ Tails: 0.5

Key Properties:

- ▶ Probabilities range from 0 to 1.
- ▶ The sum of all probabilities is 1.

Applications:

- ▶ Inventory management.
- ▶ Quality control.

A **discrete probability function** gives the probability of each specific outcome of a discrete random variable. It is called the **Probability Mass Function (PMF)**.

Characteristics

- ✓ Countable outcomes (finite or infinite)
- ✓ Probability assigned to exact values
- ✓ Sum of probabilities = 1
- ✓ Used in queue arrivals, customer counts, defects, etc.

Examples in Simulation

- ✓ Number of customers arriving per hour
- ✓ Number of system failures per day
- ✓ Number of packets in a network

Continuous Probability Function

A function that describes the probability distribution of a continuous random variable.

Example:

- ▶ Normal distribution (bell curve).

Key Properties:

- ▶ The probability of any specific value is 0.
- ▶ The total area under the curve equals 1.

Applications:

- ▶ Finance (stock prices).
- ▶ Engineering (measurement errors).

A continuous probability function describes probabilities over a continuous range of values.
It is called the Probability Density Function (PDF).

For given $f(x)$
Probability is not at a point, but over an interval:

$$P(a \leq X \leq b) = \int_a^b f(x) dx$$

Random Numbers

Numbers generated in such a way that they cannot be predicted logically.

Types:

- ▶ True Random Numbers: Generated from physical processes (e.g., atmospheric noise).
- ▶ Pseudo-Random Numbers: Generated by algorithms.

Importance in Simulation:

- ▶ Used to create randomness in models.
- ▶ Helps in Monte Carlo simulations.

Properties of Random Numbers

1. Uniform Distribution
2. Independence
3. Unpredictability
4. Reproducibility
5. Periodicity
6. Scalability
7. Randomness Tests

Pseudo- Random Numbers

These are generated by deterministic algorithms that appear random but are not and are reproducible under the same conditions.

Properties:

1. Deterministic Nature :

1. Generated based on an initial value called the seed.
2. The same seed produces the same sequence of numbers.

2. Efficiency:

1. Suitable for applications requiring large quantities of random numbers.

3. Periodicity:

1. PRNGs have a finite sequence of numbers before they repeat, known as the **period**.
2. High-quality PRNGs have a very long period to minimize repetitions.

Application

- ▶ Simulations and modeling.
- ▶ Gaming and random sampling.
- ▶ Cryptography (with additional measures).

Limitations

- ▶ Not suitable for cryptographic applications without enhancements.
- ▶ Patterns in sequences may cause biases in statistical uses.

Algorithms

- ▶ Linear Congruential Generator (LCG): Simple and widely used.
- ▶ Combined Linear Congruential Generator (LCG):

Generating Random Numbers in Simulation

Techniques:

- ▶ Linear Congruential Method.
- ▶ Combined Linear Congruential Generators.

Challenges:

- ▶ Ensuring randomness.
- ▶ Avoiding repetition.

Linear Congruential Method.

- ▶ $X_{n+1} = (a X_n + c) \bmod m$
- ▶ Where:
 - ◀ X_0 : Initial seed (starting point)
 - ◀ a : Multiplier
 - ◀ c : Increment if $c = 0$. multiplicative lcm, if $c \neq 0$, mixed lcm
 - ◀ m : Modulus
 - ◀ X_{n+1} : Next random number in the sequence

The values of a , c , and m control the randomness and cycle length of the generated numbers.

- ▶ To get a random number in the interval $[0,1)$, use:

$$R_n = X_n / m$$

Key Parameters

1. Seed (X_0): Initial value that starts the random sequence.
 2. Multiplier (a): Determines how each number influences the next.
 3. Increment (c): A constant added in each iteration (optional, for non-homogeneous generators).
 4. Modulus (m): Determines the range of values the random numbers can take.
- Note: Good parameter choices are crucial to ensure long periods and good randomness.

Example

Given values:

← $X_0=1$, $a=5$, $c=1$, $m=16$

Generate few values

← $X_1=(5 \times 1 + 1) \bmod 16 = 6$

← $X_2=(5 \times 6 + 1) \bmod 16 = 15$

← $X_3=(5 \times 15 + 1) \bmod 16 = 12$

← $X_4=(5 \times 12 + 1) \bmod 16 = 13$

(do till Cycle repeats)

DIY
 $M= 8$
 $A = 5$
 $C = 3$
 $X_0 = 0$

DIY
 $M= 16$
 $A = 5$
 $C = 1$
 $X_0 = 0$

- ▶ Period: The number of values before the sequence starts repeating.
- ▶ The length of the period is constrained by the modulus m , but it's not always m .
- ▶ Proper selection of parameters ensures the sequence has a long period.
- ▶ Example: In the previous slide, the sequence repeats after 4 steps.
- ▶ Maximizing Periodicity:
- ▶ For maximum period, the following conditions are recommended:
 - ▶ c and m should be coprime ($\gcd(c, m) = 1$). → ensures full traversal of state space
 - ▶ $a-1$ should be divisible by all prime factors of m . → prevents smaller cycles
 - ▶ a should not be congruent to 1 modulo 4 if m is a power of 2 → avoids binary correlation

Strengths and Weakness of LCG

Strengths of LCG

- ▶ Simple and Fast: LCG is computationally efficient and easy to implement.
- ▶ Deterministic: Given the same seed, it always generates the same sequence.
- ▶ Widely Used: LCG is the basis for many modern random number generators.

Weaknesses of LCG

- ▶ Periodicity Issues: If parameters are poorly chosen, the sequence can have a short period or poor randomness.
- ▶ Patterns and Correlation: Poorly chosen parameters may result in visible patterns.
- ▶ Not Cryptographically Secure: Not suitable for cryptographic purposes due to predictability.

Applications of Linear Congruential Generators

1. Monte Carlo Simulations: Used in simulations for statistical analysis.
2. Game Development: Random events and procedural generation in video games.
3. Cryptography: Although not secure, it's sometimes used in non-sensitive applications.
4. Testing and Benchmarking: Randomized tests in algorithm development.

Combined Linear Congruential Generators

- ▶ What is the Combined Linear Congruential Generator (CLCG)?
 - ▶ A method to enhance the randomness of pseudorandom number generation.
 - ▶ Combines two or more LCGs to create a more robust generator.
- ▶ Motivation for Combination:
 - ▶ A single LCG might produce numbers with undesirable correlations or short periods.
 - ▶ Combining LCGs mitigates these weaknesses by introducing diversity in the random sequences.

Formula

Suppose we have two LCGs:

- ▶ LCG 1: $X_{n+1}(1) = (a_1 X_n(1) + c_1) \bmod m_1$
- ▶ LCG 2: $X_{n+1}(2) = (a_2 X_n(2) + c_2) \bmod m_2$

Combining the Outputs:

- ▶ There are various ways to combine the outputs, such as:
 - ▶ Addition
 - ▶ XOR (exclusive OR)
 - ▶ Weighted Average

Why Combine LCGs?

- **Improve Randomness:**
 - By combining multiple LCGs, we reduce the likelihood of correlations between the numbers generated by a single LCG.
- **Increase Periodicity:**
 - The period of the combined generator can be significantly longer than the periods of individual LCGs.
- **Reduce Patterns:**
 - The combination helps avoid predictable patterns, which can arise in a single LCG sequence.

Periodicity of CLCG

- ▶ The period of the combined generator is influenced by the periods of the individual LCGs.
 - ▶ If the individual periods are coprime, the period of the combined sequence can be the least common multiple (LCM) of the individual periods.
- ▶ Example:
 - ▶ If LCG 1 has a period of 4 and LCG 2 has a period of 5, the combined period will be $\text{LCM}(4,5)=20$.
- ▶ Maximizing Periodicity:
 - ▶ Choosing parameters for LCGs that are coprime helps maximize the period of the combined generator.

Advantages and Disadvantages

Advantages:

- ▶ **Enhanced Randomness:** The combined output is less likely to exhibit patterns or correlations compared to a single LCG.
- ▶ **Longer Period:** The periods of the individual LCGs can combine to create a longer overall period.
- ▶ **Flexibility:** By using different modulus, multipliers, and increments for each LCG, the generator can be tailored to specific requirements.

Disadvantages

- ▶ **Complexity:** Implementing a combined LCG is more complex than using a single LCG.
- ▶ **Resource Intensive:** Requires more computation and memory due to the need for maintaining multiple generators.
- ▶ **Not Cryptographically Secure:** Like individual LCGs, the combined LCG may still be predictable if the parameters are known.

Other approaches

- ▶ Mersenne Twister
- ▶ XORSHIFT

Comparison?

Combined LCG vs. Mersenne Twister:

- ▶ Mersenne Twister provides better statistical properties and a very long period.
- ▶ Combined LCG offers simplicity and faster computations but may not match Mersenne Twister in randomness.

Combined LCG vs. XORSHIFT:

- ▶ Xorshift is often faster and can offer good randomness, but combining LCGs can offer a simpler implementation with certain benefits in specific use cases.

Comparison

Technique	Formula	Parameters	Period Length	Notes
Linear Congruential Generator	$X_{n+1} = (aX_n + c) \bmod m$	a, c, m, X_0	$\leq m$	Most widely used, needs good parameter selection
Mixed Generator	$X_{n+1} = (aX_n + c) \bmod m$	Same as LCG	Full period if conditions met	Better than pure LCG in period
Additive Generator	$X_n = (X_{n-j} + X_{n-k}) \bmod m$	Seeds, j, k, m	Depends on j, k, m	Better randomness, requires seed vector
Incremental Generator	$X_{n+1} = (X_n + d) \bmod m$	d, m, X_0	$\leq m$	Simple, but limited randomness

Tests for Random Number Generation

Independence Tests

- ▶ Run Test:

Evaluates the randomness of the sequence by analyzing the number of runs

Checks if the number and distribution of runs align with what is expected in a truly random sequence.

- ▶ Auto-Correlation Test:

Examines the correlation between random numbers separated by a fixed lag.

Low or no correlation indicates independence.

Uniformity Test

- ▶ Chi-square test:

Compares the distribution of the generated numbers to a uniform distribution.

- ▶ Frequency test:

Tests the randomness of a sequence of zeroes and ones.